Práctica 1
El protocolo HTTP

Humberto Martínez Barberá
Eduardo Martínez Graciá

Protocolo HTTP (1)

- La comunicación navegador – servidor web se basa en dos aspectos:
  - El protocolo HTTP
  - Cómo maneja las peticiones el servidor HTTP.
    - Si el recurso es un fichero, lo envía como respuesta
    - Si el recurso es un programa, lo ejecuta. La comunicación entre servidor web y programa se realiza mediante el Common Gateway Interface (CGI)

Protocolo HTTP (2)

- La primera versión fue la HTTP/1.0, RFC 1945
- En 1998 empezó a usarse la versión HTTP/1.1, RFC 2616
- HTTP/1.1 es compatible hacia atrás con HTTP/1.0
- Emplea TCP como protocolo de transporte
- Comunicación cliente-servidor:
  - El cliente envía un mensaje al servidor solicitando un recurso
  - El servidor recibe el mensaje, lo procesa, y responde con un mensaje HTTP de respuesta, que contiene el recurso solicitado
- El protocolo HTTP es un protocolo sin estado.

Protocolo HTTP (3)

- HTTP/1.0 usa conexiones no persistentes
- HTTP/1.1 usa conexiones persistentes (por defecto) o no persistentes
  - No persistente: cada conexión TCP se cierra después del envío
- Con conexiones **no persistentes**:

El cliente inicia una conexión TCP con el servidor www.um.es (puerto 80)

El cliente envía un mensaje de petición HTTP al servidor, incluyendo el path /diic/index.html

El servidor recibe la petición, recupera el objeto /diic/index.html y lo encapsula en el mensaje HTTP de respuesta.

El servidor cierra la conexión.

El cliente recibe el mensaje de respuesta, extrae el fichero HTML y encuentra referencias a imágenes

El cliente repite los pasos anteriores por cada imagen referenciada.

Protocolo HTTP (4)

- Los navegadores modernos pueden emplear cierto grado de paralelismo en las conexiones TCP.
- Round-trip time o RTT: tiempo que tarda un paquete pequeño en viajar desde el cliente al servidor y de vuelta al cliente.
- Cantidad de tiempo entre la petición de un fichero y su recepción: 2 RTT + tiempo de transmisión del fichero HTML
  - 1 RTT en los 2 pasos iniciales de handshake
  - 1 RTT en el envío del mensaje de petición (incluye el tercer paso del handshake) y la recepción del comienzo de la respuesta.
- Inconvenientes de las conexiones no persistentes:
  - Sobrecarga en el servidor web debido a la reserva de recursos para cada petición
  - Las implementaciones TCP comienzan con velocidades de transmisión lentas para evitar congestión inicial.

Protocolo HTTP (5)

- Conexiones **persistentes**:
  - El servidor mantiene abiertas las conexiones después de enviar la respuesta.
  - Las peticiones y respuestas entre el mismo cliente y servidor pueden ser enviadas por la misma conexión.
  - El servidor HTTP cierra las conexiones cuando no se usan durante cierto tiempo configurable.
- Tipos de conexiones persistentes
  - **Sin pipelining**: el cliente envía una nueva petición sólo cuando ha llegado la respuesta anterior.
    - Los objetos referenciados en una página sufren un RTT entre la petición y la respuesta
    - La conexión está inactiva durante cierto tiempo, consumiendo recursos en el servidor
  - **Con pipelining**: el cliente envía nuevas peticiones tan pronto encuentra referencias en los objetos previamente recibidos (antes de terminar de recibir respuestas previas). Esto reduce los RTT entre petición y respuesta.

Protocolo HTTP (6)

- Mensajes HTTP de petición. Constan de tres partes:
  - 1ª parte. **Línea de petición**
    `MétodoHTTP Identificador VersiónHTTP`

    MétodoHTTP: acción que el cliente solicita. Puede ser:

    GET, POST, HEAD, DELETE, PUT

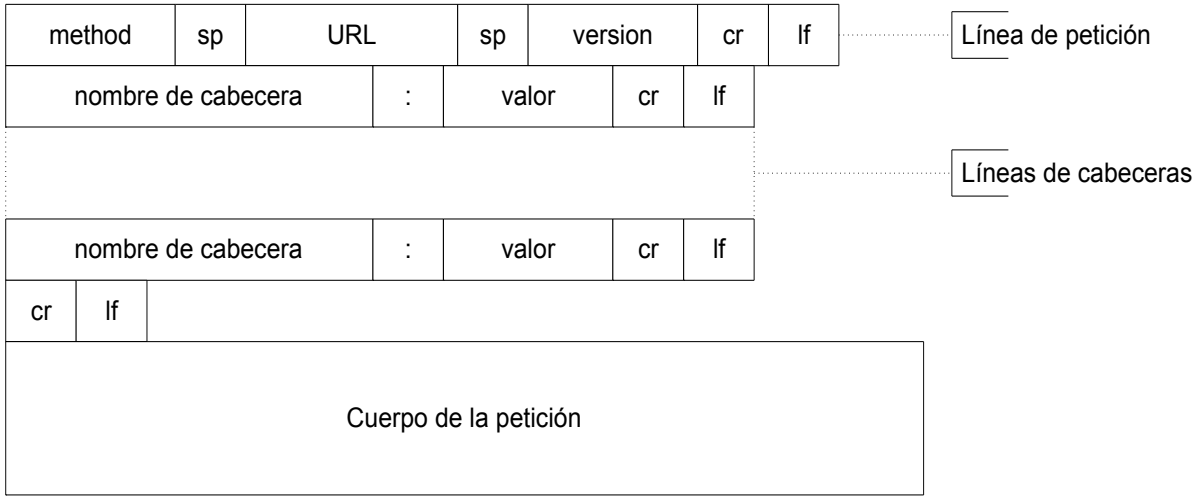
    Identificador: URL sin protocolo ni nombre del servidor

    VersiónHTTP: suele ser:

    `HTTP/1.0 ó HTTP/1.1`
  - 2ª parte. **Líneas de cabeceras de petición**
    - Informan al servidor sobre el cliente y lo que solicita.
  - 3ª parte. **Cuerpo de la petición**
    - Datos transferidos del cliente al servidor

Protocolo HTTP (7)

- Formato general de una petición HTTP y ejemplo:




```
GET /servicios/prueba.html HTTP/1.1
Host: ants.dif.um.es
Connection: close
User-agent: Mozilla/4.06
Accept-language: es
Accept: *.html
Accept: *.text
```

[Una línea en blanco, conteniendo sólo CRLF]

Protocolo HTTP (8)

- Campos de cabecera de petición comunes:
  - User-Agent: Informa del tipo de navegador que emplea el cliente
  - Referer: URL del documento desde el que se accedió al actual
  - If-Modified-Since: sólo se quiere el documento si se ha modificado
  - Accept-Language: lenguajes aceptados por el navegador
  - Host: servidor al que el cliente envía la petición
  - Connection: se emplea para mantener la conexión
  - Accept: tipos de datos aceptados por el cliente

Protocolo HTTP (9)

- Mensajes de respuesta. Constan, también, de tres partes:
  - 1ª parte. **Campo de estado**
    `VersiónHTTP CódigoEstado Explicación`

    VersiónHTTP: versión del protocolo:

    `HTTP/1.0 ó HTTP/1.1`

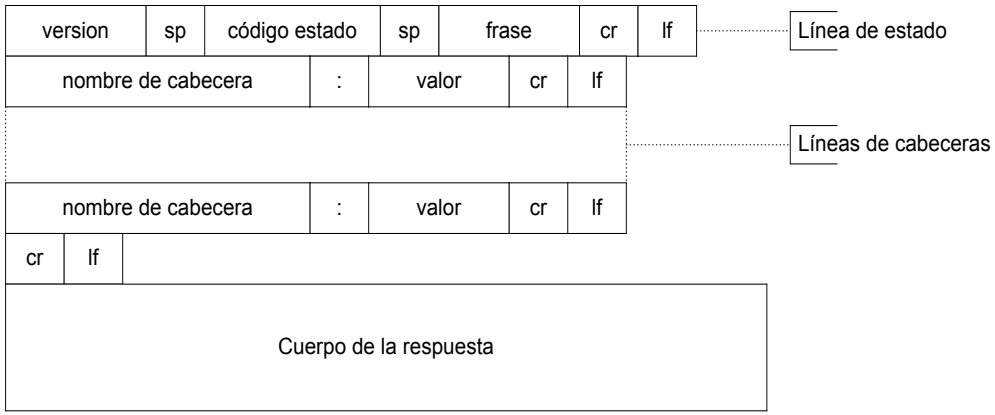
    CódigoEstado: número entre 200 y 599

    `2xx Éxito`
    `3xx Redireccionamiento`
    `4xx Error del cliente`
    `5xx Error del servidor`

    Explicación: información descriptiva del estado
  - 2ª parte. **Líneas de cabeceras de respuesta**
    - Informan al cliente sobre el servidor y el recurso consultado.
  - 3ª parte. **Cuerpo de la respuesta**
    - Datos enviados del servidor al cliente

Protocolo HTTP (10)

- Formato general de una respuesta y ejemplo:




```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 09 Mar 1999 19:49:56 GMT
Server: Apache/1.2.4
Last-Modified: Tue, 09 Mar 1999 19:44:32 GMT
Content-Length: 120
Content-Type: text/html

<html><head>
<title> Fichero HTML de prueba </title>
</head><body>
<h1>Este es un fichero de prueba</h1>
</body></html>
```

Protocolo HTTP (11)

- Campos de cabecera de respuesta comunes:
  - Location: nueva localización del documento solicitado
  - Server: nombre y versión del software servidor
  - Date: día y hora en la que se transmite el documento
  - MIME-version: versión del protocolo MIME usada por el servidor
  - Content-Length: longitud en bytes del cuerpo de la respuesta
  - Content-Type: tipo MIME que identifica el tipo de dato de la respuesta
  - Last-Modified: fecha y hora en la que se modificó por última vez
- La cabecera de respuesta la construye:
  - El servidor si el recurso solicitado es un documento
  - El programa *gateway* (Content-Type, Content-Length) y el servidor

Protocolo HTTP (12)

- Otros métodos del protocolo HTTP
  - HEAD: permite obtener información de un recurso sin recuperarlo
  - Ejemplo de petición:

```
HEAD /index.html HTTP/1.0
[CRLF]
```

  - Respuesta:

```
HTTP/1.0 200 Document follows
MIME-Version: 1.0
Server: CERN/3.0
Date: Tuesday, 09-Mar-99 20:26:27 GMT
Content-Type: text/html
Content-Length: 8969
Last-Modified: Friday, 05-Mar-99 08:31:40 GMT
[CRLF]
```

Protocolo HTTP (13)

- Otros métodos del protocolo HTTP
  - PUT y DELETE: permiten crear y eliminar recursos en el servidor
  - Ejemplo de petición PUT:

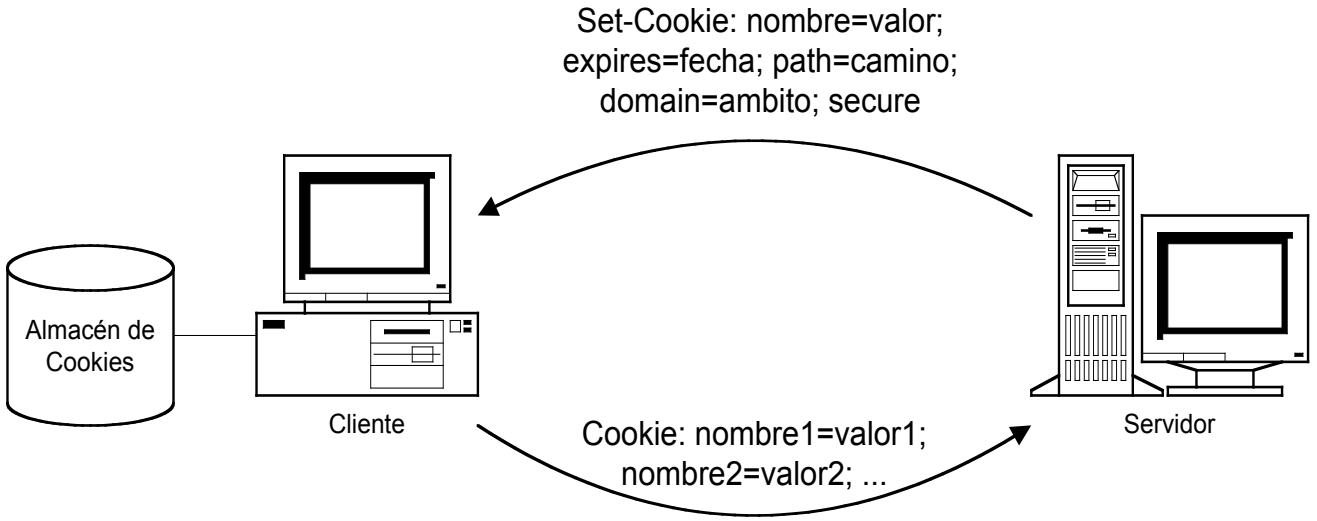
```
PUT /proyecto/dir1/fichero.html HTTP/1.0
Content-Type: text/html
Content-Length: 115
[CRLF]
<HTML><HEAD><TITLE>Fichero tonto</TITLE></HEAD>
<BODY>
<H1>Realmente, esto es una prueba tonta</H1>
</BODY></HTML>
```

  - Respuesta:

```
HTTP/1.0 201 created
[CRLF]
```

Cookies (1)

- Una cookie es información que un servidor puede enviar al cliente para que éste la almacene y se la reenvíe en posteriores accesos. La transferencia de la cookie se realiza en un mensaje de respuesta HTTP (RFC 2109).



- Aplicaciones de las cookies:
  - Recordar preferencias de un cliente para generar contenido personalizado
  - Para almacenar información de sesión (ej: carro de la compra)
  - En general: para "simular" sesiones, dado que el protocolo HTTP carece de ellas.

Cookies (2)

- Iniciación de la cookie por el servidor HTTP: insertado por un CGI en la directiva *Set-Cookie* de la cabecera de respuesta HTTP.
  - *nombre=valor* : asocia a una propiedad un valor específico.
    Espacio, coma y punto y coma se deben traducir al código URL.
  - *expires=fecha* : fecha de caducidad de la *cookie*.
    Formato: weekday, DD-Month-YY HH:MM:SS GMT
    Por defecto: al cerrar el navegador.
  - *domain=ámbito* : el cliente identifica con *ámbito* si debe enviar el cookie al servidor accedido.
    Por defecto: el servidor que genera la cookie.
  - *path=camino* : especifica los recursos a los que se envía la cookie.
    Por defecto: el script que genera la cookie.
  - *secure* : sólo se envía cuando se emplea el protocolo HTTPS
- Un servidor puede insertar varias directivas *Set-Cookie* en la cabecera
- Una *cookie* se puede eliminar indicando en *expires* una fecha pasada

Cookies (3)

- Devolución de una cookie por parte del cliente
  - Al conectarse a un servidor web, el navegador comprueba si dispone de alguna cookie con *domain* y *path* que verifican la URL consultada
  - Con las cookies que verifiquen la URL, el navegador inserta en la cabecera de petición:
    - *Cookie: nombre1=valor1; nombre2=valor2; ...*